DomesticLeadsRouter(1).js
JavaScript

domestic_leads_model(1).js
JavaScript

DomesticLeadsController(1).js
JavaScript

LeadSubmittedController.js
JavaScript

BudgetaryQuotationController.js
JavaScript

budgetary_quotation_model.js
JavaScript

BudgetaryQuotationRouter.js
JavaScript

DomesticLeadForm(1).js
JavaScript

BudgetaryQuotationForm.js
JavaScript

generateLeadNo.js
JavaScript

ExportLeadsController.js
JavaScript

CRMLeadController(1).js
JavaScript

ExportLeadForm.js
JavaScript

CRMLeadForm(1).js
JavaScript

Use these files for reference I have given backend (model, controller, router) of Domestic Lead Form and Budgetary Quotation Form as Well as the Frontend BudgetaryQuotationForm.js and DomesticLeadsForm.js Now I want you to refer these for generation of code

Domestic Leads Form generates a unique ID during the submission of form and is stored in backend db table using generateLeadNo logic (i have attached that as well please refer) Now I want to use this unique ID leadReferenceNo and autofill the Matching fields in the Budgetary Quotation Form Also there must be one dropdown in Budgetary Quotation form which shows the unique ID leadReferenceNo of all three Domestic Leads Form Export Leads Form and CRM Lead Form and as soon as I click on the particular option (leadReferenceNo) I must be able to autofill all the matching fields
This is my requirement Please give me code to complete my requirement refer the files I have given
Give me a complete guide where I must add the part of code or replace to achieve this in detail guide I require

Great — your requirement is **very clear and achievable**, and your existing structure is already strong 👍
I'll give you a **complete step-by-step implementation guide (backend + frontend)** using your current files.

🎯 WHAT YOU WANT (Simplified)

1. Domestic / Export / CRM → already generate/store leads
2. Each has **leadReferenceNo (unique ID)**
3. In **BudgetaryQuotationForm**:
   - Add **dropdown of all leadReferenceNo**
   - When selected → **auto-fill matching fields**

⚠️ IMPORTANT OBSATION (from your code)

- Domestic uses `leadReferenceNo` ✅ domestic_leads_model
- BudgetaryQuotation uses `referenceNo` ❗ budgetary_quotation_model

  👉 We will map:

```
leadReferenceNo → referenceNo
```

✅ STEP 1 — BACKEND: CREATE COMMON API (VERY IMPORTANT)

🎯 Goal:

Fetch **ALL leads (Domestic + Export + CRM)** in one API

📁 Create NEW controller:

👉 `LeadReferenceController.js`

```
import db from "../models/index.js";

const Domestic = db.DomesticLeadsModel;
const Export = db.ExportLeadsModel;
const CRM = db.CRMLeadsModel;

export const GetAllLeadReferences = async (req, res) => {
  try {
    const domestic = await Domestic.findAll({ raw: true });
    const exportLeads = await Export.findAll({ raw: true });
    const crm = await CRM.findAll({ raw: true });

    // Normalize structure
    const formattedDomestic = domestic.map(d => ({
      leadReferenceNo: d.leadReferenceNo,
      tenderName: d.tenderName,
      customerName: d.customerName,
      customerAddress: d.customerAddress,
      leadOwner: d.leadOwner,
      source: "DOMESTIC"
    }));

    const formattedExport = exportLeads.map(d => ({
      leadReferenceNo: d.leadReferenceNo,
      tenderName: d.tenderName,
      customerName: d.customerName,
      customerAddress: d.customerAddress,
      leadOwner: d.leadOwner,
      source: "EXPORT"
    }));

    const formattedCRM = crm.map(d => ({
      leadReferenceNo: d.leadID, // CRM uses leadID
      tenderName: d.tenderName,
      customerName: d.organisation,
      customerAddress: "",
      leadOwner: "",
      source: "CRM"
```

```
    }));

    const allLeads = [
      ...formattedDomestic,
      ...formattedExport,
      ...formattedCRM
    ];

    res.status(200).json({
      success: true,
      data: allLeads
    });

  } catch (error) {
    console.error(error);
    res.status(500).json({
      success: false,
      message: "Error fetching lead references"
    });
  }
};
```

📁 Create Router:

👉 `LeadReferenceRouter.js`

```
import { GetAllLeadReferences } from "../controllers/LeadReferenceController.js";

export const LeadReferenceRouter = (app) => {
  app.get("/getAllLeadReferences", GetAllLeadReferences);
};
```

JavaScript

📁 Register Router in server:

👉 In your `server.js` or `index.js`

```
import { LeadReferenceRouter } from "./routes/LeadReferenceRouter.js";
```

JavaScript

✅ STEP 2 — FRONTEND (BudgetaryQuotationForm.js)

File: BudgetaryQuotationForm

2.1 Add state

JavaScript

2.2 Fetch lead references

Inside `useEffect`:

```
axios.get(nodeIp + "/getAllLeadReferences")
  .then(res => {
    if (res.data.success) {
      setLeadOptions(res.data.data);
    }
  })
  .catch(err => console.log(err));
```

2.3 Add Dropdown in UI

Add this inside your form:

```
<Controller
  name="referenceNo"
```

JavaScript

```
      {...field}
      select
      fullWidth
      label="Select Lead Reference"
    >
      {leadOptions.map((lead, index) => (
        <MenuItem key={index} value={lead.leadReferenceNo}>
          {lead.leadReferenceNo} ({lead.source})
        </MenuItem>
      ))}
    </TextField>
  )}
/>
```

🔹 2.4 Auto-fill logic ⭐ (MOST IMPORTANT)

Add this function:

```
const handleLeadSelect = (selectedRefNo) => {
  const selectedLead = leadOptions.find(
    (lead) => lead.leadReferenceNo === selectedRefNo
  );

  if (selectedLead) {
    reset({
      referenceNo: selectedLead.leadReferenceNo,
      bqTitle: selectedLead.tenderName || "",
      customerName: selectedLead.customerName || "",
      customerAddress: selectedLead.customerAddress || "",
      leadOwner: selectedLead.leadOwner || "",
    });
  }
};
```

🔹 2.5 Attach onChange

Modify dropdown:

```
onChange={(e) => {
  field.onChange(e);
```

```
handleLeadSelect(e.target.value);
```

JavaScript

✅ STEP 3 — OPTIONAL (Better UX)

Show extra info:

```
<MenuItem key={index} value={lead.leadReferenceNo}>
  {lead.leadReferenceNo} - {lead.customerName} ({lead.source})
```

JavaScript

✅ STEP 4 — IMPORTANT FIELD MAPPING

Lead Form Field	BQ Field
leadReferenceNo	referenceNo
tenderName	bqTitle
customerName	customerName
customerAddress	customerAddress
leadOwner	leadOwner

✅ STEP 5 — WHY THIS WORKS

- Domestic leads already store unique ID using `generateUniqLeadNo()` generateLeadNo
- You fetch all leads via one API
- Dropdown → select → auto-fill

🚀 FINAL RESULT

✔ Dropdown shows all leads (Domestic + Export + CRM)
✔ Selecting one → auto-fills form
✔ Works dynamically with DB

🔥 BONUS (VERY IMPORTANT IMPROVEMENT)

Later you can:

- Add **searchable dropdown (Autocomplete)**
- Add **filter by source (Domestic/Export/CRM)**
- Lock `referenceNo` after selection

❓ If you want next

I can:
✅ Generate **FULL updated BudgetaryQuotationForm.js ready-to-run**
✅ Add **search + filter UI**
✅ Add **edit mode auto-fill from table**

Just tell 👍

Sources

BudgetaryQuotation uses referenceNo ❗
👉 We will map:

No referenceNo is different It is tender Reference No we need not map it

Ahh perfect — this is an **important clarification**, and it actually changes the design in a better way 👍

You're absolutely right:

👉 `referenceNo` **(BQ) = Tender Reference No**
👉 `leadReferenceNo` **= Unique tracking ID (system-generated)**

❌ So we should **NOT map them**
✅ Instead → we should **ADD leadReferenceNo into BQ**

🎯 UPDATED APPROACH (Correct Architecture)

✅ What we will do:

1. Keep `referenceNo` as-is (Tender Ref No)
2. Add new field:

JavaScript

✅ STEP 1 — BACKEND CHANGES

📁 Update Model

File: budgetary_quotation_model

🔧 Add new column

```
leadReferenceNo: {
  type: Sequelize.STRING,
```

JavaScript

👉 Add this **above or below** `referenceNo`

📁 Update Controller

File: BudgetaryQuotationController

🔧 Add to Create API

JavaScript

So it becomes:

```
const BudgetaryQuotationModelEx = {
  bqTitle: req.body.bqTitle,
  customerName: req.body.customerName,
  customerAddress: req.body.customerAddress,
  leadOwner: req.body.leadOwner,

  leadReferenceNo: req.body.leadReferenceNo, // ✅ NEW

  defenceAndNonDefence: req.body.defenceAndNonDefence,
  estimateValueInCrWithoutGST: req.body.estimateValueInCrWithoutGST,
```

```
  submittedValueInCrWithoutGST: req.body.submittedValueInCrWithoutGST,
  dateOfLetterSubmission: req.body.dateOfLetterSubmission,
  referenceNo: req.body.referenceNo,
  ...
};
```

✅ STEP 2 — FRONTEND CHANGES (BudgetaryQuotationForm)

File: BudgetaryQuotationForm

🔹 2.1 Add default value

```
defaultValues: {
  leadReferenceNo: "",    // ✅ NEW
  referenceNo: "",
  ...
}
```

🔹 2.2 Add state

JavaScript

🔹 2.3 Fetch leads (same as before)

```
axios.get(nodeIp + "/getAllLeadReferences")
  .then(res => {
    if (res.data.success) {
      setLeadOptions(res.data.data);
    }
  });
```

🔹 2.4 Add Dropdown (IMPORTANT)

```
<Controller
  name="leadReferenceNo"
  control={control}
  render={({ field }) => (
    <TextField
      {...field}
      select
      fullWidth
      label="Select Lead Reference No"
      onChange={(e) => {
        field.onChange(e);
        handleLeadSelect(e.target.value);
      }}
    >
      {leadOptions.map((lead, index) => (
        <MenuItem key={index} value={lead.leadReferenceNo}>
          {lead.leadReferenceNo} ({lead.source})
        </MenuItem>
      ))}
    </TextField>
  )}
/>
```

🔹 2.5 Auto-fill Logic

```
const handleLeadSelect = (selectedRefNo) => {
  const selectedLead = leadOptions.find(
    (lead) => lead.leadReferenceNo === selectedRefNo
  );

  if (selectedLead) {
    reset({
      leadReferenceNo: selectedLead.leadReferenceNo,

      // autofill only matching fields
      bqTitle: selectedLead.tenderName || "",
      customerName: selectedLead.customerName || "",
      customerAddress: selectedLead.customerAddress || "",
      leadOwner: selectedLead.leadOwner || "",

      // ❌ DO NOT TOUCH
      referenceNo: "",   // Tender ref stays manual
    });
  }
};
```

🔹 2.6 Send to backend

Modify submit:

```
const formattedData = {
  leadReferenceNo: data.leadReferenceNo, // ✅ NEW

  bqTitle: data.bqTitle,
  customerName: data.customerName,
  customerAddress: data.customerAddress,
  leadOwner: data.leadOwner,
  referenceNo: data.referenceNo,
  ...
};
```

✅ FINAL BEHAVIOR (Correct Flow)

🧠 User Flow:

1. User opens BQ form
2. Selects **LeadReferenceNo dropdown**
3. System:
   - Fetches lead data
   - Autofills:
     - tenderName → bqTitle
     - customerName
     - address
     - leadOwner
4. User manually enters:
   - Tender Reference No (referenceNo)

🔥 WHY THIS IS BEST DESIGN

✔ Keeps **business meaning correct**
✔ No confusion between IDs
✔ Supports full pipeline tracking:

```
Lead → BQ → Tender → Order
          ↑
   leadReferenceNo
```

🚀 NEXT LEVEL (Recommended)

If you want pro-level system:

⭐ Add validation:

```
if (!data.leadReferenceNo) {
  alert("Please select Lead Reference No");
  return;
}
```

JavaScript

⭐ Lock autofilled fields (optional)

JavaScript

⭐ Show Lead Preview Card

(after selection)

❓ Want next upgrade?

I can now:

✅ Generate **FULL ready-to-run BudgetaryQuotationForm.js (cleaned + integrated)**
✅ Add **searchable dropdown (Autocomplete)**
✅ Link BQ → Tender → Order tracking using same ID

Just say 👍

Sources

Give me a better guide all changes in Budgetary, Domestic, Frontend backend all in one guide

Perfect — now I'll give you a **clean, end-to-end MASTER GUIDE** (no confusion, production-ready approach) covering:

✔ Backend (ALL modules)
✔ Frontend (ALL forms)
✔ Data flow
✔ Exact code placement

🚀 FINAL ARCHITECTURE (VERY IMPORTANT)

🎯 Core Idea:

```
leadReferenceNo = MASTER TRACKING ID (used everywhere)
referenceNo     = Tender Reference (separate)
```

🧠 FINAL FLOW

```
Domestic / Export / CRM → generate leadReferenceNo
                ↓
Budgetary Quotation → SELECT leadReferenceNo → autofill
                ↓
Future modules (Tender, Order, etc)
```

==========

✅ STEP 1 — BACKEND (LEADS)

==========

📁 Domestic Leads (Already Correct)

✔ You already generate ID using:
generateLeadNo

👉 Just confirm inside controller:

File: DomesticLeadsController

✅ Ensure this exists in Create API:

```
const leadReferenceNo = await generateUniqLeadNo(DomesticLeadsModel);

const DomesticLeadsModelData = {
  leadReferenceNo,  // ✅ MUST EXIST
```

```
  ...
};
```

📁 Export Leads (FIX REQUIRED ❗)

File: `ExportLeadsController`

❌ Problem:

No `leadReferenceNo`

✅ Fix:

```
const leadReferenceNo = await generateUniqLeadNo(ExportLeadsModel);

const ExportLeadData = {
  leadReferenceNo,   // ✅ ADD THIS
  ...
};
```

📁 CRM Leads (IMPORTANT FIX ❗)

File: `CRMLeadController`

❌ Problem:

Uses `leadID` instead of unified ID

✅ Fix (BEST PRACTICE)

```
const leadReferenceNo = await generateUniqLeadNo(CRMLeadsModel);

const CRMLeadsModelEx = {
  leadReferenceNo,   // ✅ NEW COMMON FIELD
  leadID: req.body.leadID, // keep if needed
  ...
};
```

==========

✅ STEP 2 — COMMON API (CRITICAL)

==========

📁 Create NEW file:

`LeadReferenceController.js`

```
import db from "../models/index.js";

export const GetAllLeadReferences = async (req, res) => {
  try {
    const domestic = await db.DomesticLeadsModel.findAll({ raw: true });
    const exportLeads = await db.ExportLeadsModel.findAll({ raw: true });
    const crm = await db.CRMLeadsModel.findAll({ raw: true });

    const allLeads = [
      ...domestic.map(d => ({
        leadReferenceNo: d.leadReferenceNo,
        tenderName: d.tenderName,
        customerName: d.customerName,
        customerAddress: d.customerAddress,
        leadOwner: d.leadOwner,
        source: "DOMESTIC"
      })),
      ...exportLeads.map(d => ({
        leadReferenceNo: d.leadReferenceNo,
        tenderName: d.tenderName,
        customerName: d.customerName,
        customerAddress: d.customerAddress,
        leadOwner: d.leadOwner,
        source: "EXPORT"
      })),
      ...crm.map(d => ({
        leadReferenceNo: d.leadReferenceNo,
        tenderName: d.tenderName,
        customerName: d.organisation,
        customerAddress: "",
        leadOwner: "",
        source: "CRM"
      }))
    ];

    res.json({ success: true, data: allLeads });

  } catch (err) {
    res.status(500).json({ success: false, error: err });
  }
};
```

📁 Router

```
export const LeadReferenceRouter = (app) => {
  app.get("/getAllLeadReferences", GetAllLeadReferences);
```

JavaScript

📁 Register

JavaScript

==========

✅ STEP 3 — BUDGETARY BACKEND

==========

File: budgetary_quotation_model

🔧 Add new field

```
leadReferenceNo: {
  type: Sequelize.STRING,
```

JavaScript

📁 Controller update

File: BudgetaryQuotationController

JavaScript

==========

✅ STEP 4 — FRONTEND (BQ FORM)

==========

File: BudgetaryQuotationForm

🔹 4.1 Add State

JavaScript

🔹 4.2 Fetch Leads

Inside `useEffect`:

```
axios.get(nodeIp + "/getAllLeadReferences")
  .then(res => {
    if (res.data.success) {
      setLeadOptions(res.data.data);
    }
  });
```

🔹 4.3 Add Dropdown

```
<Controller
  name="leadReferenceNo"
  control={control}
  render={({ field }) => (
    <TextField
      {...field}
      select
      fullWidth
      label="Lead Reference No"
      onChange={(e) => {
        field.onChange(e);
        handleLeadSelect(e.target.value);
      }}
    >
      {leadOptions.map((lead, index) => (
```

```
        <MenuItem key={index} value={lead.leadReferenceNo}>
          {lead.leadReferenceNo} ({lead.source})
        </MenuItem>
      ))}
    </TextField>
  )}
/>
```

🔹 4.4 Autofill Logic ⭐

```
const handleLeadSelect = (refNo) => {
  const lead = leadOptions.find(l => l.leadReferenceNo === refNo);

  if (lead) {
    reset({
      leadReferenceNo: lead.leadReferenceNo,
      bqTitle: lead.tenderName || "",
      customerName: lead.customerName || "",
      customerAddress: lead.customerAddress || "",
      leadOwner: lead.leadOwner || "",
    });
  }
};
```

🔹 4.5 Default Values

```
defaultValues: {
  leadReferenceNo: "",
  referenceNo: "",
```

🔹 4.6 Submit

```
const formattedData = {
  leadReferenceNo: data.leadReferenceNo,
  referenceNo: data.referenceNo,
  ...
};
```

==========

✅ STEP 5 — FINAL RESULT

==========

🎯 What you achieved:

- ✔ Unified tracking ID across system
- ✔ Dropdown from 3 modules
- ✔ Autofill working
- ✔ No confusion with tender reference

🔥 BONUS (HIGHLY RECOMMENDED)

⭐ Add Autocomplete (Better UX)

Replace dropdown with:

```
<Autocomplete
  options={leadOptions}
  getOptionLabel={(opt) => `${opt.leadReferenceNo} - ${opt.customerName}`}
  onChange={(e, val) => handleLeadSelect(val?.leadReferenceNo)}
  renderInput={(params) => <TextField {...params} label="Select Lead" />}
/>
```

🚀 NEXT LEVEL FEATURES (If you want)

I can help you build:

- ✔ Full **Lead → BQ → Tender tracking dashboard**
- ✔ KPI cards clickable (you asked earlier)
- ✔ Audit trail using same leadReferenceNo
- ✔ Smart filtering (Domestic / Export / CRM tabs)

⚡ If you want FULL CODE

Say:

👉 "*generate full ready-to-run files*"

I'll give:

- Updated Controllers
- Models
- Routers
- Frontend complete files

No debugging needed 🚀

Sources